

FINAL PROJECT

TOPIC: TIMER IP DESIGN

Student: Phan Thanh Binh

Class: IC31

Instructor: Tri Le

Contents

Chapter 1: Overview	1
1.1 Introduction	1
Chapter 2: System Design.....	2
2.1 Functional requirements.....	2
2.2 Detail Functional Description.....	3
2.2.1 APB Slave / Register	3
2.2.2 Counter	3
2.2.3 Halted mode.....	4
2.2.4 Timer interrupt.....	4
2.2.5 Counting Mode.....	4
2.3 Interface signals.....	4
2.4 Register specification.....	6
2.4.1 Register summary.....	6
2.4.2 TCR Register	7
2.4.3 TDR0	8
2.4.4 TDR1	8
2.4.5 TCMP0	8
2.4.6 TCMP1	9
2.4.7 TIER	9
2.4.8 TISR.....	10
2.4.9 THCSR	11
2.5 Design block diagram.....	12
2.6 Design detail.....	14
2.6.1 APB Slave.....	14
2.6.2 Register.....	16
2.6.3 Counter Control	24
2.6.4 Counter	26
2.6.5 Interrupt	28

Chapter 3: Design Verification	30
---	----

Chapter 1: Overview

1.1 Introduction

- Timer is an essential module for every chip
- This is used to generate accurate timing interval or controlling the timing of various operations within the circuit. Timer can be used in various application: pulse generation, delay generation, event generation, PWM generation, interrupt generation
- In this project, a timer module is customized from CLINT module of industrial RISC-V architecture. It is used to generate interrupt based on user settings.

Chapter 2: System Design

2.1 Functional requirements

This Timer IP design have following features:

- 64-bit count-up
- 12-bit address
- Register set is configured via APB bus (IP is APB slave)
- Support wait state (1 cycle is enough) and error handling
- Support halt (stop) in debug mode
- Timer uses active low async reset
- Counter can be counted based on system clock or divided up to 256
- Support timer interrupt (can be enabled or disabled)

2.2 Detail Functional Description

2.2.1 APB Slave / Register

- Read/write to reserved area is RAZ/WI (read as zero, write ignored)
- Support byte access: bus can access to individual bytes in the register.
- Support wait state (1cycle) to improve the timing.
- Support error handling for some prohibited access:
 - + write prohibited value to *TCR.div_val*
 - + *div_en* or *div_val* changes during timer is operating
- When error occurs, data is not written into register bit/fields

2.2.2 Counter

- 64-bit count-up.
- Counting mode:
 - + Default mode: counter's speed is same as system clock.
 - + Control mode: when enabled by writing 1 to *TCR.div_en* bit, the counter's speed is determined by the divisor value set in *TCR.div_val*
- Counter continues counting when interrupts occurs.
- Counter continues counting when overflow occurs.
- Support halted mode describe in next page.
- When *timer_en* changes from H->L, the counter is cleared to its initial value by timer itself (hardware). When *timer_en* is L->H again, timer can work normally.
- The *div_en* and *div_val* is not related to frequency divisor (clock divider). Those settings only control the counter when to count.

2.2.3 Halted mode

- Counter can be halted (stopped) in debug mode when both below conditions occur:
 - + Input debug_mode signal is High,
 - + THCR.halt_req is 1.
- THCSR.halt_ack is 1 after a halt request indicates that the request is accepted.
- After halted, counter can be resumed to count normally when clearing the halt request to 0.
- The period of each counting number needs to be same when halt and not halt.

2.2.4 Timer interrupt

- Timer interrupt (tim_int) is asserted (set) when interrupt is enabled and counter's value matches (equal) the compare value.
- Once asserted, the timer interrupt (tim_int) remains unchanged until it is cleared by writing 1 to *TISR.int_st* bit or the interrupt is disabled.

2.2.5 Counting Mode

- In default mode, counting speed depends on system clock (same as div_val=0 case as below waveform).
- The counting's speed can be controlled by register settings by setting into TCR.div_en and TCR.div_val[3:0] as the register specification
- div_en and div_val can not be changed during timer_en is High.
- Not allow to change div_en and div_val while timer_en is High by an error response when user mistakenly accesses.

2.3 Interface signals

Interface signal is described as table 1

Table1: Interface signals of timer IP

Signal name	Width	Direction	Description
sys_clk	1	Input	System Clock
sys_rst_n	1	Input	System reset
tim_psel	1	Input	Indicates that the Slave is selected and a data transfer is required.
tim_pwrite	1	Input	An APB write access when is HIGH and an APB read access when is LOW
tim_penable	1	Input	Indicates the second and subsequent cycles of an APB transfer.
tim_paddr[11:0]	12	Input	Is the APB address bus
tim_pwdata[31:0]	32	Input	Write data bus is driven during write cycles when tim_pwrite is HIGH
tim_prdata[31:0]	32	Output	Read data bus is driven during read cycles when tim_pwrite is LOW
tim_pstrb[3:0]	4	Input	Byte lanes to update during a write transfer, not be active during a read transfer
tim_pready	1	Output	Slave signals ready; can extend the transfer
tim_pslverr	1	Output	An optional signal that can be asserted HIGH by the Master to indicate an error condition
tim_int	1	Output	Is asserted when the interrupt is enabled and counter's value matches the compare value
dbg_mode	1	Input	For simple, dbg_mode does not change after timer_en is High.

2.4 Register specification

2.4.1 Register summary

Address	Abbreviation	Register name
0x00	TCR	Timer Control Register
0x04	TDR0	Timer Data Register 0
0x08	TDR1	Timer Data Register 1
0x0C	TCMP0	Timer Compare Register 0
0x10	TCMP1	Timer Compare Register 1
0x14	TIER	Timer Interrupt Enable Register
0x18	TISR	Timer Interrupt Status Register
0x1C	THCSR	Timer Halt Control Status Register
Others	Reserved	

2.4.2 TCR Register

Bit	Name	Type	Default value	Description
31:12	Reserved	-	20'b0	Reserved
11:8	div_val	RW	4'b0001	Counter control mode setting: - 4'b0000: Counting speed is not divided (default) - 4'b0001: Counting speed is divided by 2 - 4'b0010: Counting speed is divided by 4 - 4'b0011: Counting speed is divided by 8 - 4'b0100: Counting speed is divided by 16 - 4'b0101: Counting speed is divided by 32 - 4'b0110: Counting speed is divided by 64 - 4'b0111: Counting speed is divided by 128 - 4'b1000: Counting speed is divided by 256 - Others: reserved, prohibit settings. (*)Add hardware logic to ensure div_val is prohibited to change when timer_en is high. Access is error response in this case (*)Access is “error response” when setting prohibit value to div_val
7:2	Reserved	RO	6'b0	Reserved
1	div_en	RW	1'b0	Counter control mode enable. - 0:Disabled. Counter counts with normal speed based on system clock - 1:Enabled. The counting speed of counter is controlled based on div_val

				(*) add hardware logic to ensure div_en is prohibited to change when timer_en is High. Access is error reponse in this case
0	timer_en	RW	1'b0	Timer enable - 0: Disabled. Counter does not count - 1: Enabled. Counter starts counting (*) timer_en changes from H->L will initialize the TDR0/1 to their initial value

2.4.3 TDR0

Bit	Name	Type	Default value	Description
31:0	TDR0	RW	32'h0000_0000	Lower 32-bit of 64-bit counter (*) Value of this register is cleared to initial value when timer_en changes from H->L

2.4.4 TDR1

Bit	Name	Type	Default value	Description
31:0	TDR1	RW	32'h0000_0000	Upper 32-bit of 64-bit counter (*) Value of this register is cleared to initial value when timer_en changes from H->L

2.4.5 TCMP0

Bit	Name	Type	Default value	Description
31:0	TCMP0	RW	32'hFFFF_FFFF	Lower 32-bit of 64-bit compare value Interrupt peding bit is asserted when counter value is qual to compare value

2.4.6 TCMP1

Bit	Name	Type	Default value	Description
31:0	TCMP1	RW	32'hFFFFFF_FFFF	Upper 32-bit of 64-bit compare value Interrupt pending bit is asserted when counter value is equal to compare value

2.4.7 TIER

Bit	Name	Type	Default value	Description
31:1	Reserved	RO	31'h0	Reserved
0	int_en	R/W	1'b0	- Timer interrupt enable + 0: Timer interrupt is disabled. + 1: Timer interrupt is enabled. - When this bit is 0, no timer interrupt is output. - When this bit is 1, timer interrupt pending bit can be output (interrupt pending bit is set when counter value is equal to compare value). - Clearing this bit to 0 while interrupt is asserting will mask the interrupt to 0 but does not affect the interrupt pending bit (TISR.int_st bit).

2.4.8 TISR

Bit	Name	Type	Default value	Description
31:1	Reserved	RO	31'h0	Reserved
0	int_st	RW1C	1'b0	- Timer interrupt trigger condition status bit (interrupt pending bit) + 0: the interrupt trigger condition does not occur. + 1: the interrupt trigger condition occurred. - Write 1 when this bit is 1 to clear it to 0. - Write 0 when this bit is 1 has no effect - Write to this bit when it is 0 has no effect. Note1: When interrupt trigger condition occurred (counter reached compare value), counter continues to count normally. Note2: the clearance condition has highest priority.

2.4.9 THCSR

Bit	Name	Type	Default value	Description
31:2	Reserved	RO	30'h0	Reserved
1	halt_ack	RO	1'b0	Timer halt acknowledge 0: timer is NOT halted 1: timer is halted Timer accepts the halt request only in debug mode, indicates by debug_mode input signal
0	halt_req	RW	1'b0	Timer halt request 0: no halt req. 1: timer is requested to halt.

2.5 Design block diagram

Based on the above functions, we can divide them into the following blocks: APB, Register, Counter control, Counter, Interrupt. The block diagram is shown in the figure 2.1.

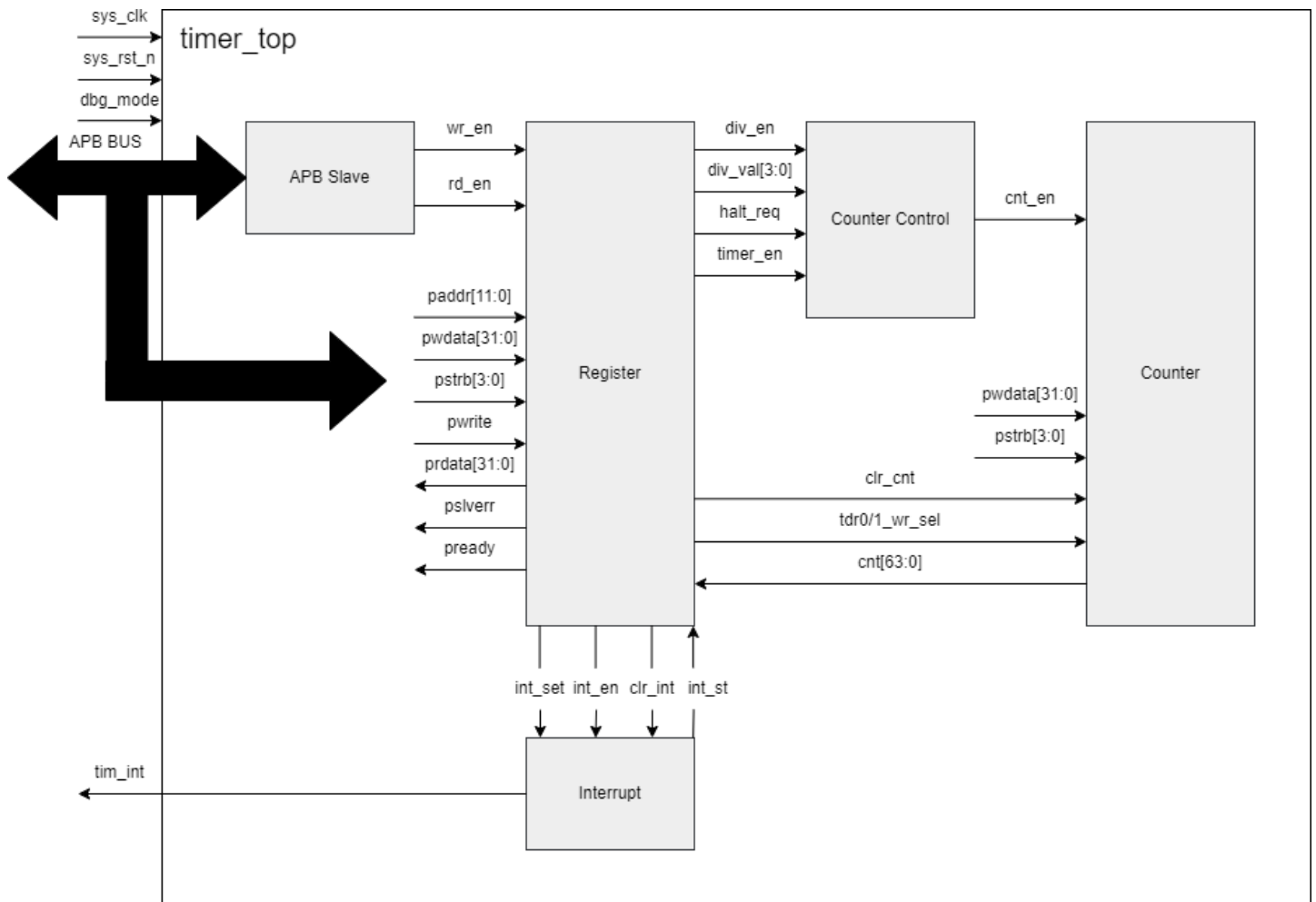


Figure 2.1: Block diagram

- APB Slave block: The APB Slave block provides the interface between the APB bus and the timer module. It decodes read and write transactions from the bus and generates control signals (`wr_en`, `rd_en`) to enable register access. Its purpose is to ensure correct handshaking and communication with the APB bus.

- Register block: This block stores configuration and control registers, such as divider value, halt request, and timer enable. It manages data flow between the APB interface and internal logic.

- Interrupt block: Responsible for generating timer interrupts (tim_int) when programmed conditions are met.

- Counter control block: The Counter Control block determines when the counter should increment. It generates the cnt_en signal based on the selected divider, timer enable, and halt request. Its purpose is to provide flexible timing control and support different counting modes depending on the configuration.

- Counter: The Counter block is a 64-bit up-counter that increments on every active cnt_en pulse. It can be accessed and updated through the APB interface, allowing both software read/write operations and automatic counting. Its purpose is to serve as the core timing element of the module.

2.6 Design detail

2.6.1 APB Slave

The APB slave module implements the control logic for an AMBA APB (Advanced Peripheral Bus) slave interface. Its main responsibilities include handling read/write enable signals and generating the pready signal according to the APB protocol timing. This APB slave has below features:

- Supports APB protocol (setup & access phases).
- Generates wr_en for write operations.
- Generates rd_en for read operations.
- Provides pready signal to indicate transfer completion.
- Active-low reset and synchronous operation with pclk.

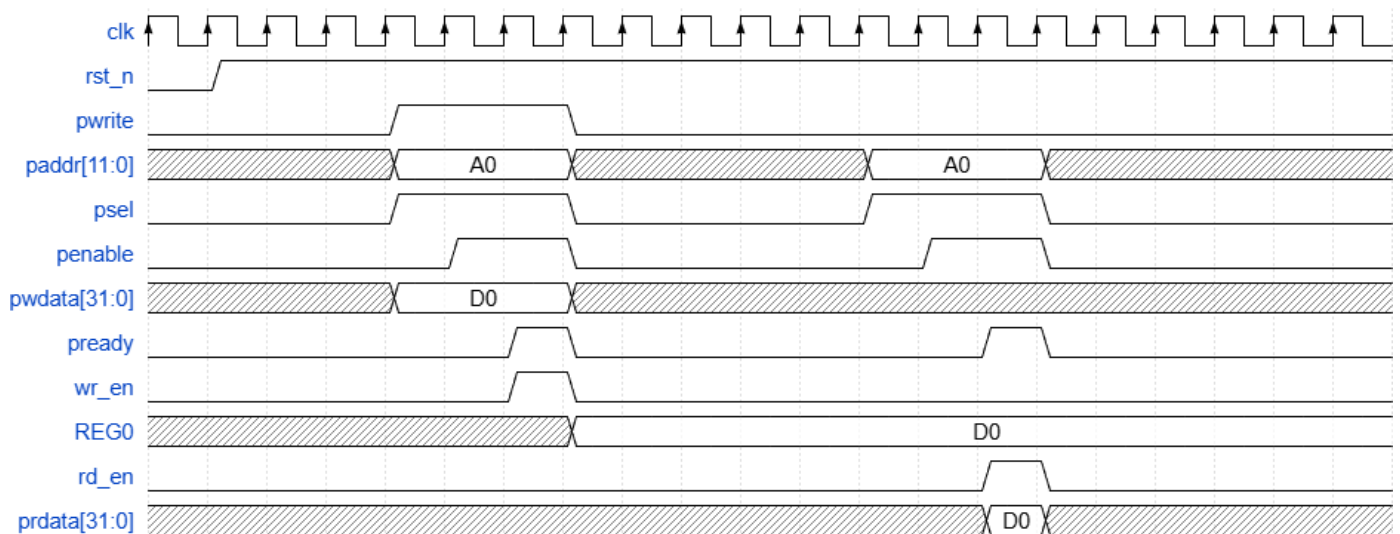


Figure 2.2: Waveform of APB Slave

Design of APB Slave is described as figure below

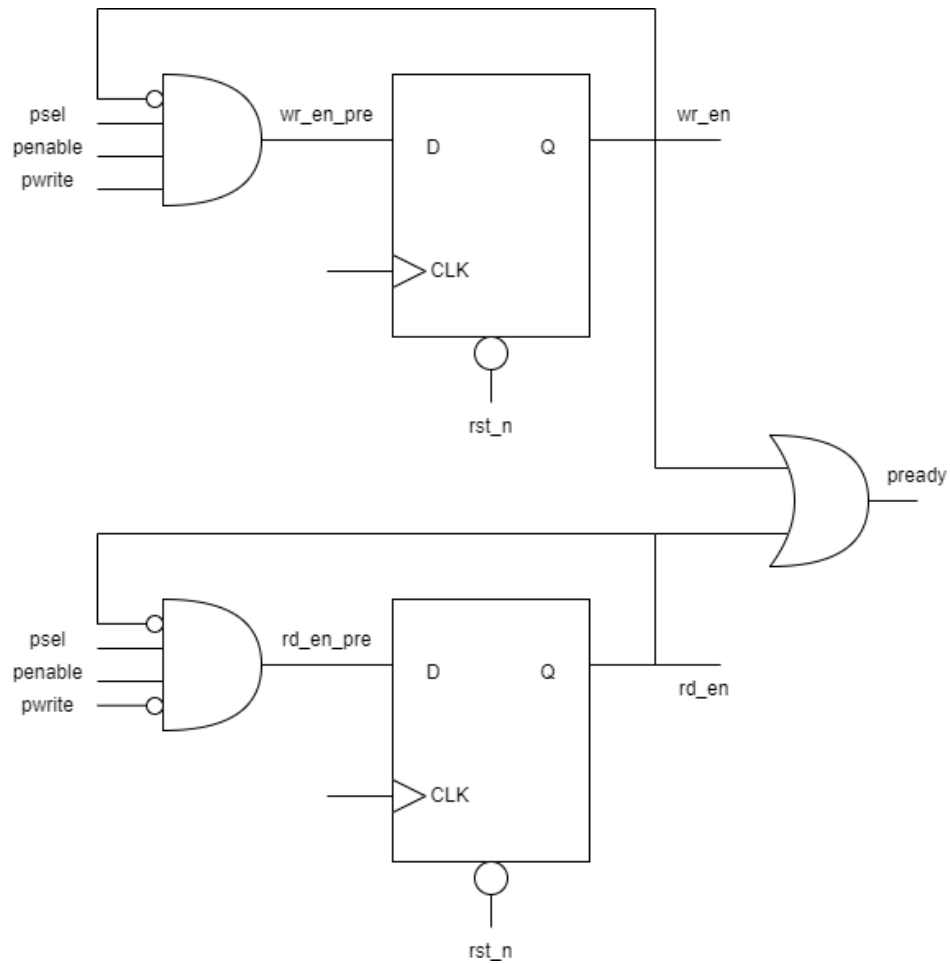


Figure 2.3: Design of APB Slave

- The wr_en signal is a pulse and is asserted when psel, penable and pwrite is High. It indicates that there's a write command. This signal is sent to register module to write data into register.

- The rd_en signal is a pulse and is asserted when psel, penable is High. It indicates that there's a read command. This signal is sent to the register module to read data from the register.

- The PREADY signal is generated by a logical OR operation between wr_en and rd_en, indicating that the peripheral is ready to complete either a read or write transaction in the current cycle.

2.6.2 Register

The register module implements the control and status registers of a 64-bit timer. It interfaces with the APB bus through read/write signals and a 12-bit address decoder. The registers allow software to configure timer operation, enable interrupts, read the current counter value, and control debug-related functions. This register and interrupt has below features:

- Supports a 64-bit counter (accessible through TDR0 and TDR1)
- Includes 64-bit compare registers (TCMP0/TCMP1) for match detection.
- Generates an interrupt on counter/compare match when enabled.
- Provides debug halt control for stopping the timer in debug mode.
- Implements error checking to prevent invalid register updates.

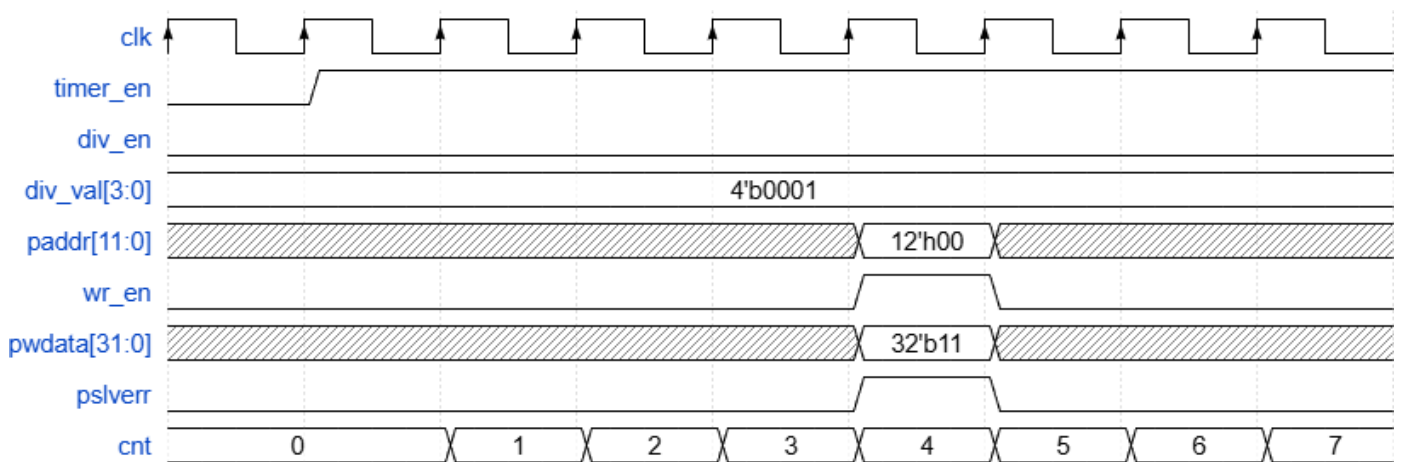


Figure 2.4: Waveform of TCR register

The image illustrates the scenario where timer_en is activated and the div_en signal changes.

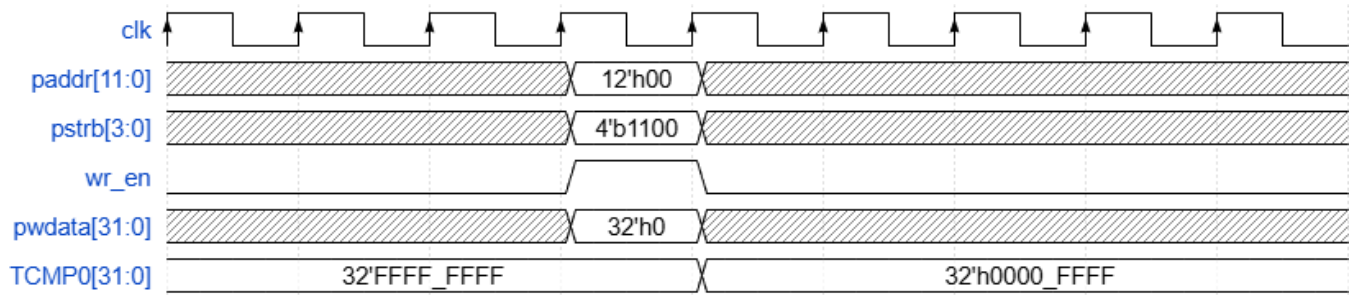


Figure 2.5: Waveform of TCMP register

The figure illustrates sparse data writes to the TCMP register. When pstrb = 4'b1100, only byte3, byte2 are updated, while byte1, byte0 retain their previous values and remain unaffected.

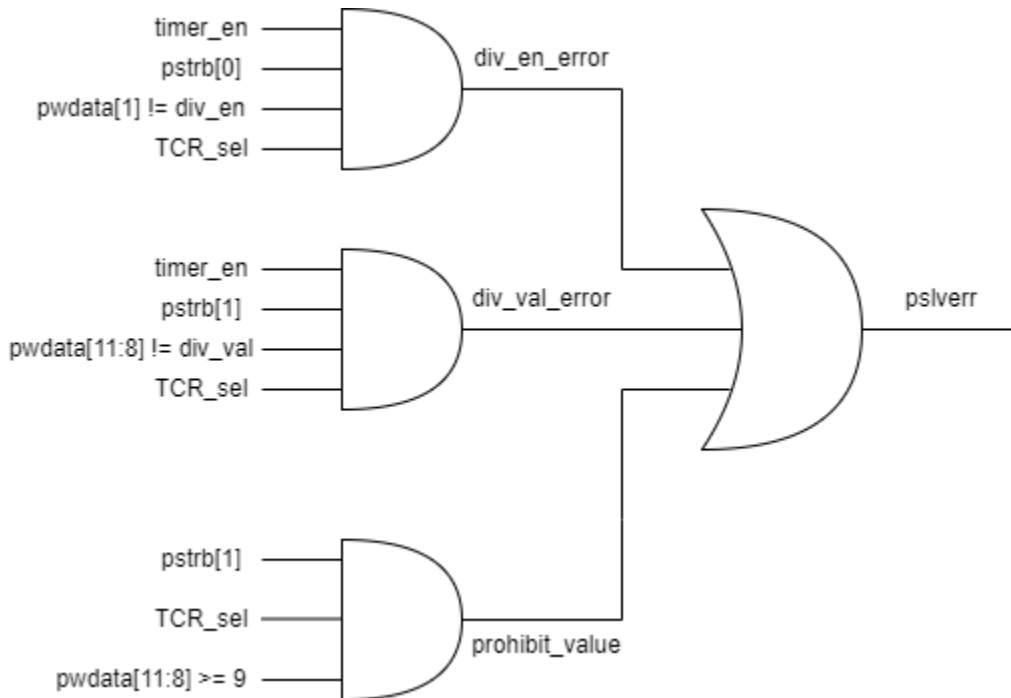


Figure 2.6: Logic diagram for pslverr

The figure above shows the logic diagram for error detection when div_en or div_val is modified while timer_en is asserted, and when an invalid value is written to div_val.

- div_en_error: This logic detects an error when div_en is changed while timer_en is enabled. This error condition is generated as a combinational logic of TCR_sel, timer_en, pstrb[0] (since div_en is located in byte0), and the condition that the value written on pwwdata[1] differs from the current div_en value stored in the TCR register.

- div_val_error: This logic is similar to div_en_error. In this case, div_val is located in byte1; therefore, div_val_error is generated as the logical AND of TCR_sel, pstrb[1], timer_en, and the condition that pwwdata[11:8] differs from the current div_val stored in the TCR register.

- prohibit_value: This condition occurs when the user writes an invalid value to div_val. The prohibit_value error is generated as a combinational logic of TCR_sel, pstrb[1], and the condition $pwwdata[11:8] \geq 9$.

- pslverr: The output pslverr shall be asserted when any one of the above error conditions occurs. When pslverr is asserted, the write operation to the TCR register shall be suppressed.

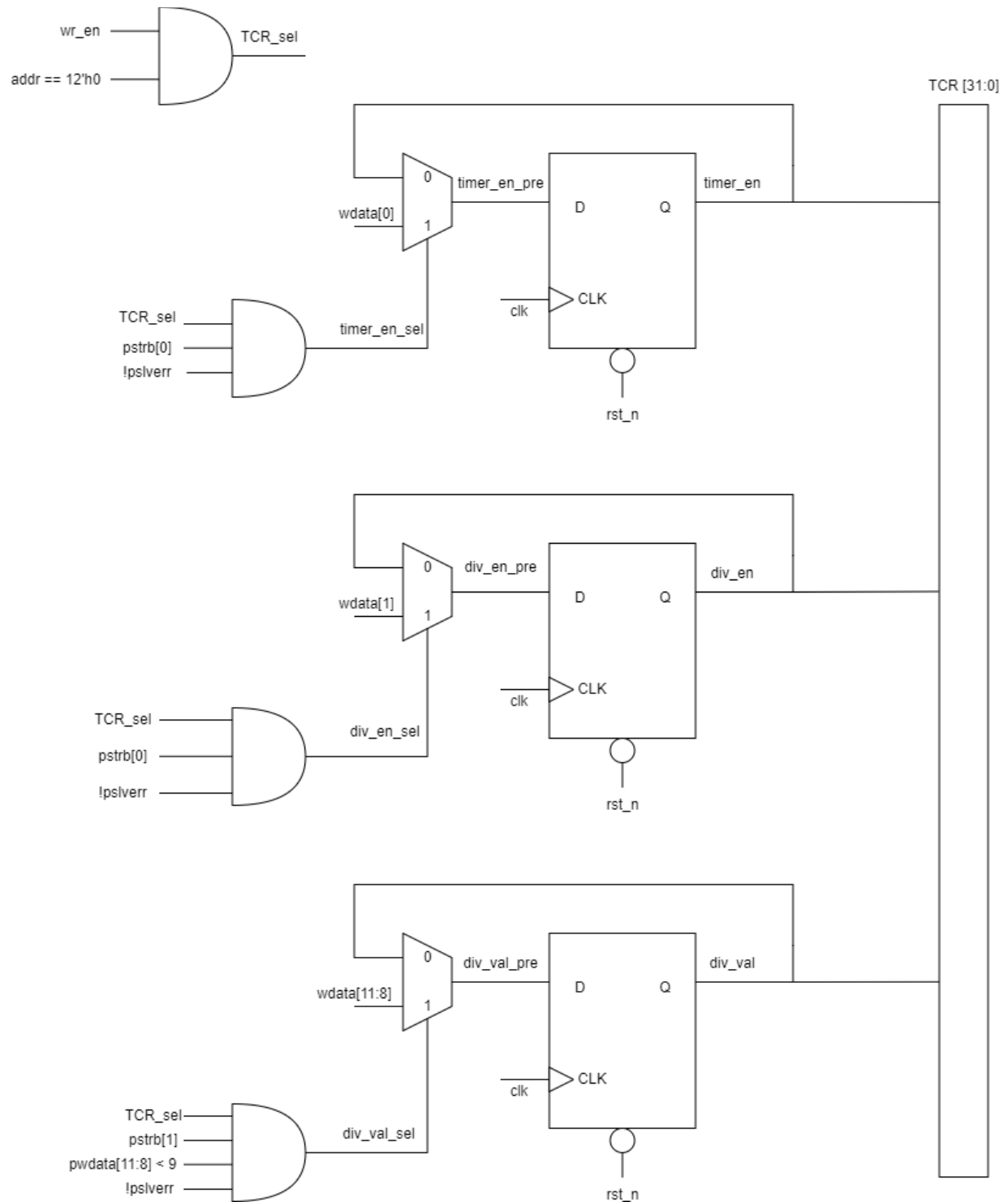


Figure 2.7: Logic diagram for TCR register

In the TCR register logic diagram shown above, flip-flops are used for timer_en, div_en, and div_val to support both write operations and value storage.

- timer_en: Indicates whether the timer is running when asserted high (1). timer_en shall be updated when a write to the TCR register is selected (TCR_wr_sel) and the write transaction contains no error. If pwdata[0] = 1, timer_en shall be set (the counter runs). Conversely, if pwdata[0] = 0, timer_en shall be cleared (the counter stops).

- div_en: Enables frequency division for the counting operation; the division behavior depends on the value stored in TCR. div_en shall be set by writing to the TCR register without error with pwdata[1] = 1.

- div_val: Specifies the division value used for frequency division. To write div_val, the TCR register shall be selected for write, and the field pwdata[11:8] shall be within the valid range 4'b0000 to 4'b1000.

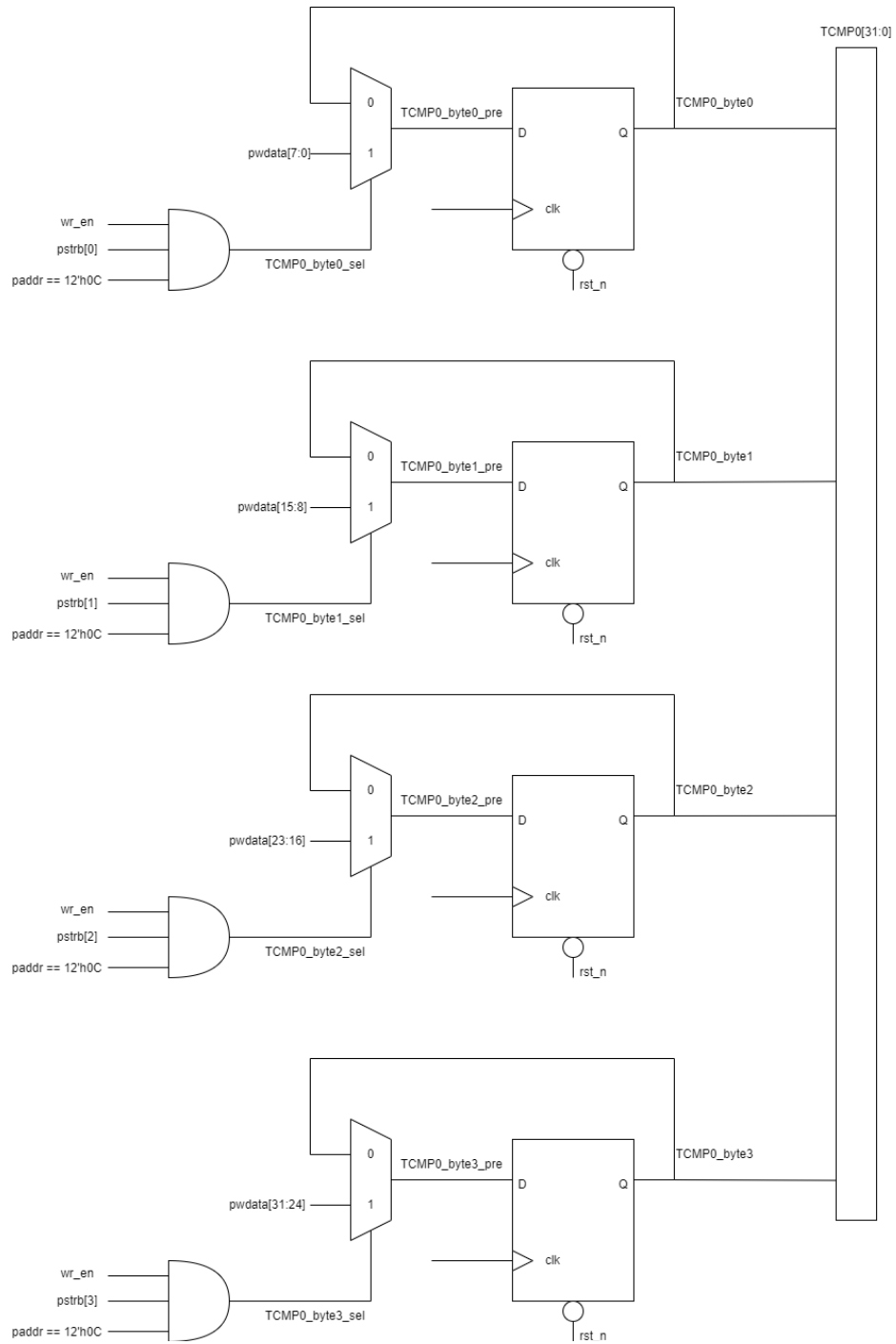


Figure 2.8: Logic diagram for TCMP register

The figure above illustrates sparse data writes to the TCMP0 register. Data is written on a byte basis according to the pstrb signal, where pstrb[0:4] correspond to byte0–byte4, respectively.

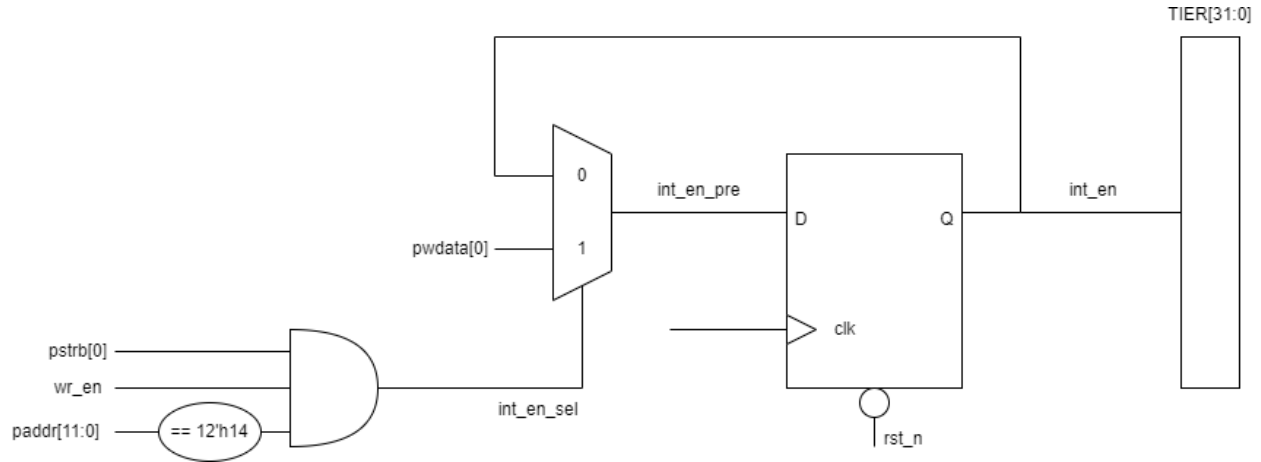


Figure 2.9: Logic diagram for TIER

To enable the interrupt, data shall be written to the TIER register. The interrupt is enabled when pstrb[0] = 1 and pwrdata[0] = 1, and disabled when pwrdata[0] = 0.

2.6.3 Counter Control

The counter control logic is used to ensure correct operation of the main counter in frequency-division mode. In addition, the counter control logic ensures that the debug mode operates in accordance with the specified requirements..

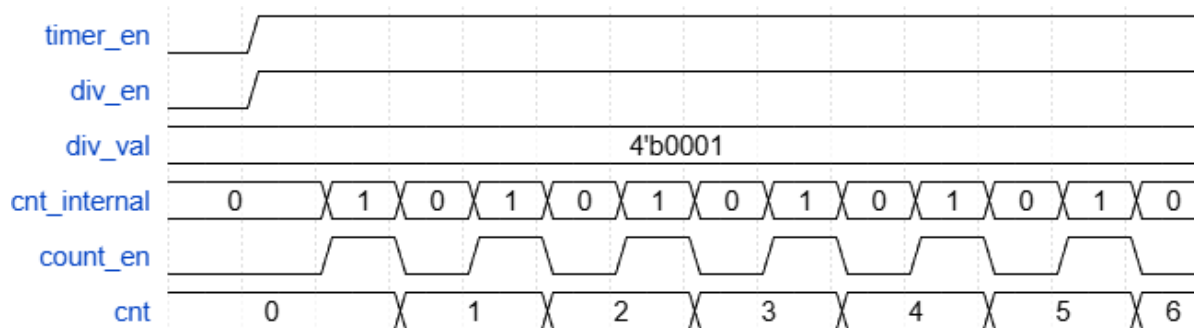


Figure 2.12: Waveform of counter control

The figure above illustrates the operation of the counter control logic with $div_val = 4'b0001$. The counter control counter counts over two values, 0 and 1. When the count reaches 1, the `count_en` signal is asserted. The main counter increments based on the assertion of `count_en`.

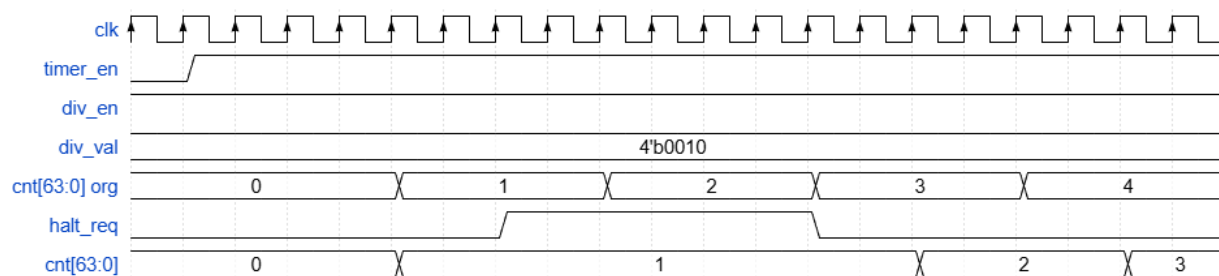


Figure 2.13: Waveform of halt mode

The figure above illustrates the counter operation when the halt signal is asserted. In this scenario, div_val is set to $4'b0010$. When the halt signal is asserted, `cnt[63:0]` holds its value at 1, while `cnt_internal` has already counted two clock cycles. After `halt_req` is deasserted, the counter resumes normal operation. Once two additional clock cycles are received, the main counter increments by one.

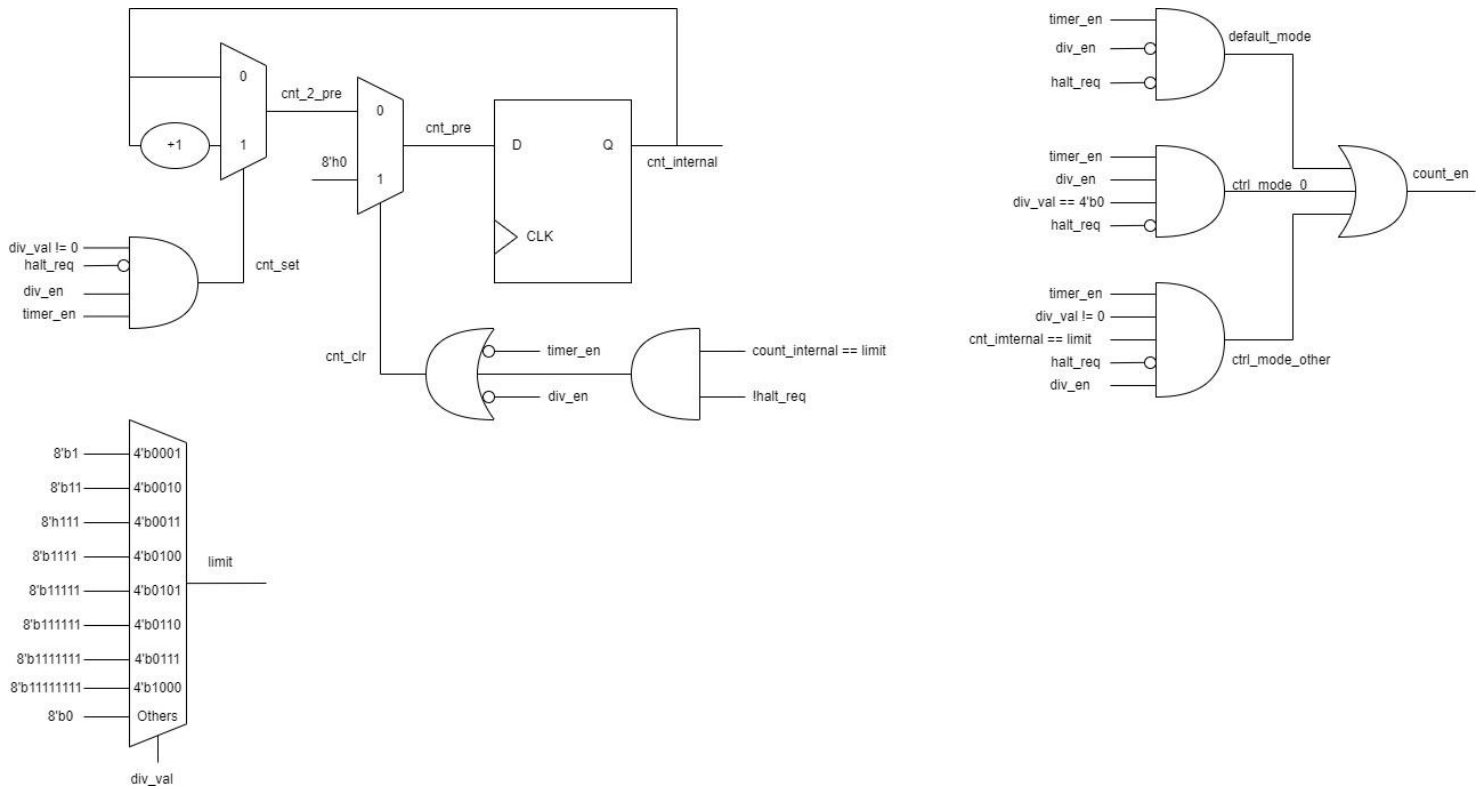


Figure 2.14: Logic diagram for Counter control

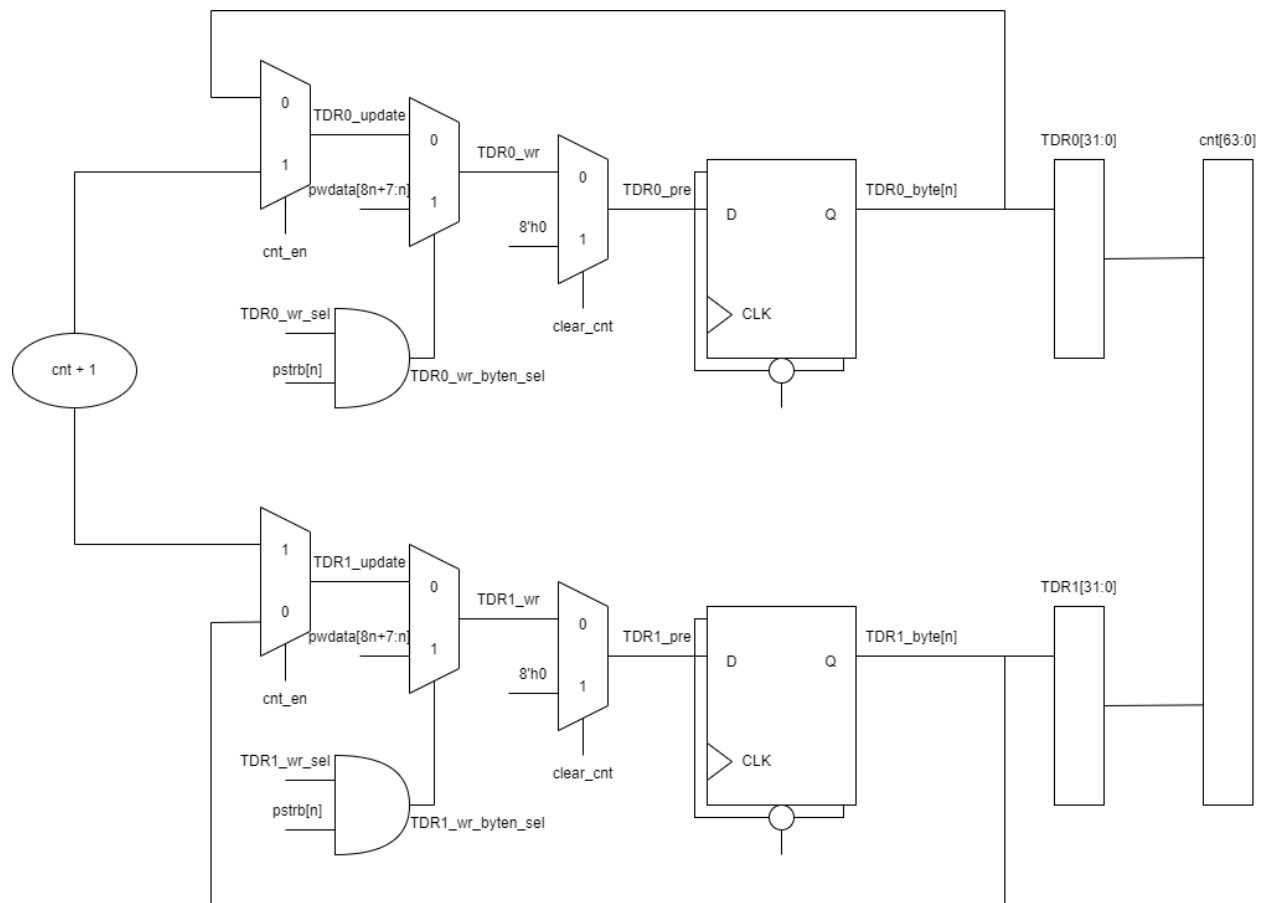
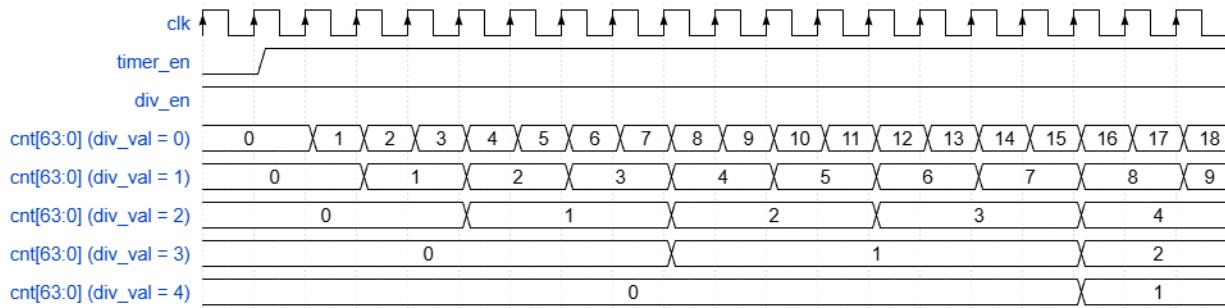
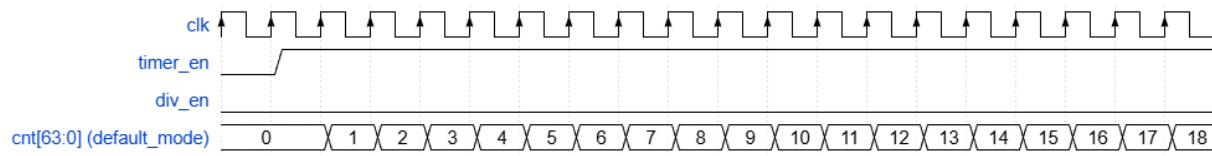
- `default_mode`: This mode is active when `div_en` is not enabled. When `timer_en` = 1, `div_en` = 0, and the system is not in halt mode, the `default_mode` signal remains asserted, allowing the counter to operate directly on the clock (`clk`).

- `ctrl_mode_0`: This mode is active when `div_en` is enabled and `div_val` = 4'b0000, and therefore behaves similarly to `default_mode`. `ctrl_mode_0` is generated as the logical AND of `timer_en`, `div_en`, `div_val` == 4'b0000, and `halt_req` = 0.

- `ctrl_mode_other`: This mode is active when `div_en` is enabled and `div_val` ≠ 4'b0000. `ctrl_mode_other` is asserted when `timer_en`, `div_en`, `halt_req` = 0, `div_val` ≠ 4'b0000, and `cnt_internal` == `limit` are all satisfied.

- `cnt_en`: The `cnt_en` signal is generated as the logical OR of `default_mode`, `ctrl_mode_0`, and `ctrl_mode_other`.

2.6.4 Counter



In the logic diagram shown above, when a clear counter condition occurs (timer_en transitions from high to low), the TDR0 and TDR1 registers shall be reset to 32'h0, resulting in the counter value being reset to 64'h0.

When no clear condition is present, if a new value is to be written to TDR0 or TDR1, the corresponding register address shall be selected and the target byte(s) shall be written using the appropriate pstrb signals. After the write operation completes, the counter shall resume counting from the newly written value.

When neither a clear condition nor a write operation is present, and cnt_en is asserted, the counter logic shall increment normally.

2.6.5 Interrupt

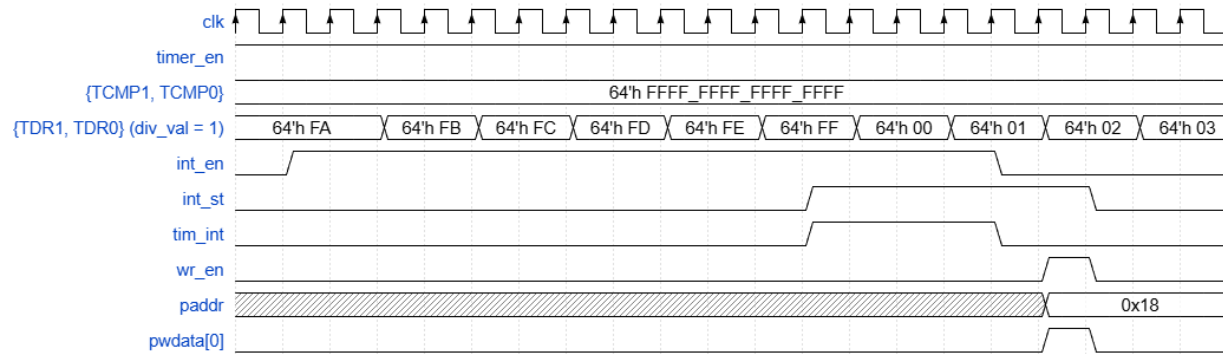


Figure 2.18: Waveform for interrupt

In the waveform, when the counter value matches the compare value in TCMP, the interrupt signal is asserted. At this time, int_en is enabled, therefore the interrupt is propagated to the output tim_int. Subsequently, int_en is deasserted, and as a result, the interrupt is no longer driven to the tim_int output. The interrupt status int_st remains asserted and is only cleared when a value of 1 is written to the TISR register.

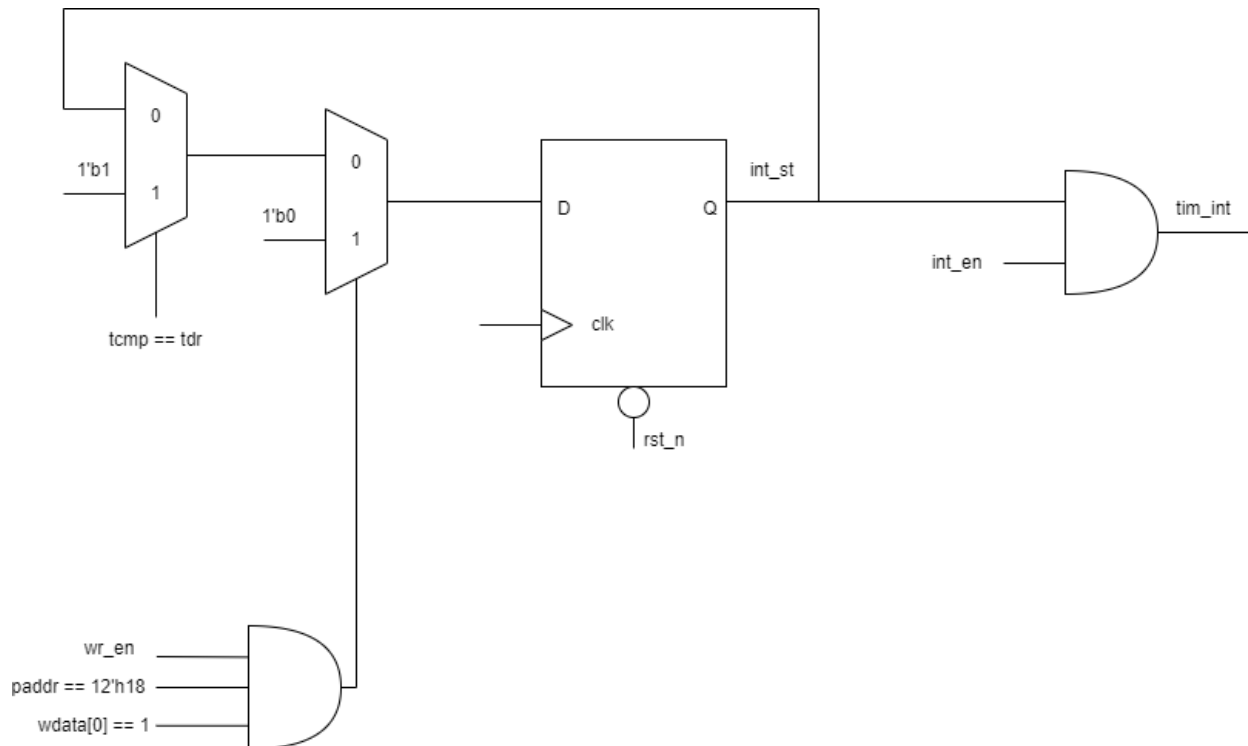


Figure 2.19: logic diagram for interrupt

When the counter value equals TCMP, a logic ‘1’ is latched into int_st on the next clock edge. The output tim_int is generated as the logical AND of int_st and int_en; therefore, the interrupt output is asserted only when both signals are logic ‘1’. When an interrupt occurs, the interrupt status is retained in the register. To clear the interrupt status, a logic ‘1’ shall be written to the TISR register.

Chapter 3: Design Verification

The test scenarios are listed in detail in the test plan as follows:

https://docs.google.com/spreadsheets/d/1IIW3XJ_vSJqfHSszgEXk_UcbO97cPovV/edit?usp=sharing&ouid=115148239021928587382&rtpof=true&sd=true